MODUL 24 · BAB VI · ALGORITMA GRAF

Algoritma Dijkstra

Lintasan terpendek dari satu sumber untuk bobot nonnegatif: selalu selesaikan simpul yang jaraknya paling kecil lebih dulu.

TUJUAN PEMBELAJARAN

Setelah mempelajari modul ini, kamu diharapkan mampu:

- Menjelaskan langkah-langkah algoritma Dijkstra serta peran himpunan S dan antrean Q.
- Menelusuri Dijkstra iterasi demi iterasi dan menyusun lintasan terpendek dari predecessor.
- Menjelaskan mengapa Dijkstra benar untuk bobot nonnegatif dan memberi contoh kegagalannya pada bobot negatif.
- Menganalisis running time Dijkstra untuk beberapa implementasi antrean prioritas.
- Membandingkan Dijkstra dengan Bellman-Ford, BFS, dan Prim.

PRASYARAT

- Modul 22: jarak terpendek, INITIALIZE-SINGLE-SOURCE, RELAX.
- Modul 23 (disarankan): Bellman-Ford sebagai pembanding.
- Modul 13: konsep Greedy.

BAB / MATERI ASAL

Materi 13.2 SINGLE SOURCE SHORTEST PATHS, Algoritma Dijkstra

POSISI DALAM URUTAN BELAJAR

Modul terakhir. Menutup Bab VI dan menghubungkan konsep Greedy, antrean prioritas, dan relaksasi.

DAFTAR ISI

1. Menyelesaikan yang Terdekat Lebih Dulu 1
2. Pseudocode Dijkstra 2
3. Penelusuran Dijkstra dari Simpul s 2
4. Mengapa Dijkstra Benar, dan Kapan Gagal 5
5. Running Time dan Perbandingan 6
6. Kesimpulan 7
- Kuis Pemahaman 9
- Kunci Jawaban dan Pembahasan 10
- Referensi 10

PENDAHULUAN

1. Menyelesaikan yang Terdekat Lebih Dulu

Bellman-Ford (Modul 23) merelaksasi semua busur berulang kali tanpa memedulikan urutan, sehingga running time-nya $\Theta(VE)$. Jika semua bobot **tidak negatif**, kita bisa jauh lebih cerdas. Pikirkan: simpul yang saat ini memiliki estimasi jarak terkecil tidak mungkin dicapai lebih murah lewat simpul lain, karena setiap jalan memutar hanya menambah bobot (atau tidak mengubahnya). Jadi jaraknya sudah **final**, dan kita cukup merelaksasi busur keluar dari simpul itu **sekali saja**.

Inilah gagasan **algoritma Dijkstra** (Dijkstra, 1959). Ia adalah algoritma Greedy (Modul 13): di setiap langkah, ia "mengunci" simpul dengan jarak sementara terkecil. Hasilnya sama dengan Bellman-Ford, tetapi

jauh lebih cepat. Syaratnya tegas: **tidak boleh ada bobot busur yang negatif**.

Modul ini membahas pseudocode Dijkstra dari bahan kuliah, menelusurinya pada graf lima simpul, menjelaskan kebenarannya, memberi contoh ketika syarat bobot nonnegatif dilanggar, dan menganalisis running time-nya.

PEMBAHASAN INTI

2. Pseudocode Dijkstra

DIJKSTRA(G, w, s)

```
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  S = ∅
3  Q = G.V
4  while Q ≠ ∅
5      u = EXTRACT-MIN(Q)
6      S = S ∪ {u}
7      for each vertex v ∈ G.Adj[u]
8          RELAX(u, v, w)
```

Parameter masukan: graf berarah berbobot G yang direpresentasikan sebagai adjacency-list, bobot w setiap busur (semuanya ≥ 0), dan simpul awal s. INITIALIZE-SINGLE-SOURCE dan RELAX sama persis dengan yang dipakai Bellman-Ford (Modul 22).

Langkah	Penjelasan
1	Inisialisasi: setiap simpul $v.d = \infty$ dan $v.\pi =$ NIL; $s.d = 0$.
2	S dikosongkan. S berisi simpul-simpul yang jarak terpendeknya dari s **sudah final**.
3	Q berisi semua simpul; Q adalah antrean prioritas minimum berdasarkan $v.d$.
4–8	Selama Q tidak kosong, jalankan langkah 5–8.
5	Keluarkan simpul u dengan $u.d$ terkecil dari Q. Pada iterasi pertama, simpul itu adalah s.
6	Masukkan u ke S: jarak u kini final.
7–8	Untuk setiap v dalam $Adj[u]$, panggil RELAX: jika $v.d > u.d + w(u, v)$, perbarui $v.d$ dan $v.\pi = u$.

Tabel 1. Penjelasan langkah-langkah Dijkstra.

CONTOH DAN PENERAPAN

3. Penelusuran Dijkstra dari Simpul s

Contoh dari bahan kuliah: tentukan lintasan terpendek dari s ke simpul-simpul lain. Busur dan bobot: $(s, t) = 10$, $(s, y) = 5$, $(t, x) = 1$, $(t, y) = 2$, $(x, z) = 4$, $(y, t) = 3$, $(y, x) = 9$, $(y, z) = 2$, $(z, s) = 7$, $(z, x) = 6$. Adjacency-list:

s	t	x	y	z
t → y	x → y	z	x → z → t	s → x

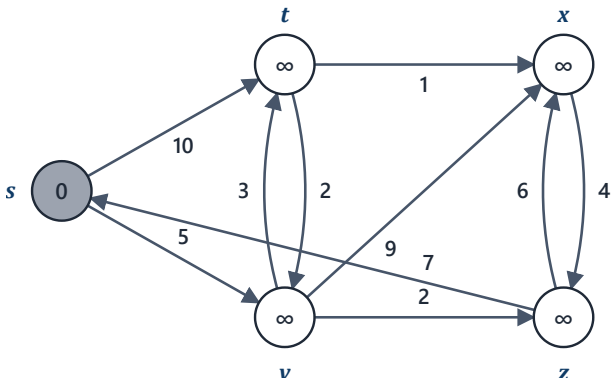
Tabel 2. Adjacency-list graf contoh.

Langkah 1. $s.d = 0$, simpul lain ∞, semua predecessor NIL. **Langkah 2.** $S = \emptyset$. **Langkah 3.** $Q = \{s, t, x, y, z\}$.

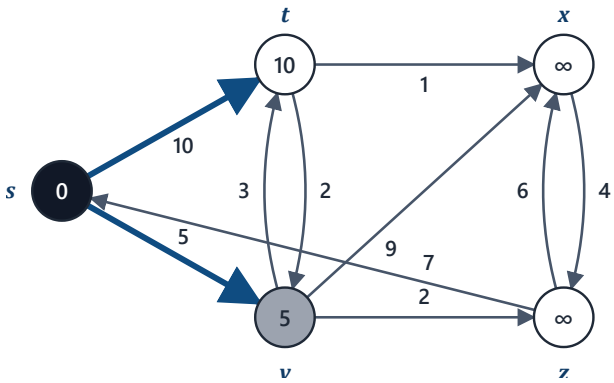
Loop	Extract u	S / Q setelah langkah 6	$Adj[u]$ dan hasil RELAX	Min di Q berikutnya
1	s (0)	{s} / {t,x,y,z}	t: 10 (π = s); y: 5 (π = s)	y (5)
2	y (5)	{s,y} / {t,x,z}	x: 14 (π = y); z: 7 (π = y); t: 10 → 8 (π = y)	z (7)
3	z (7)	{s,y,z} / {t,x}	s: tetap 0 ($0 < 7 + 7$); x: 14 → 13 (π = z)	t (8)
4	t (8)	{s,y,z,t} / {x}	x: 13 → 9 (π = t); y: tetap 5	x (9)
5	x (9)	{s,y,z,t,x} / { }	z: tetap 7 ($7 < 9 + 4$)	-
6	Langkah 4: Q kosong, kondisi FALSE, end while. Pseudocode Dijkstra selesai.			

Tabel 3. Setiap iterasi while: simpul yang di-extract, isi S dan Q, serta hasil RELAX.

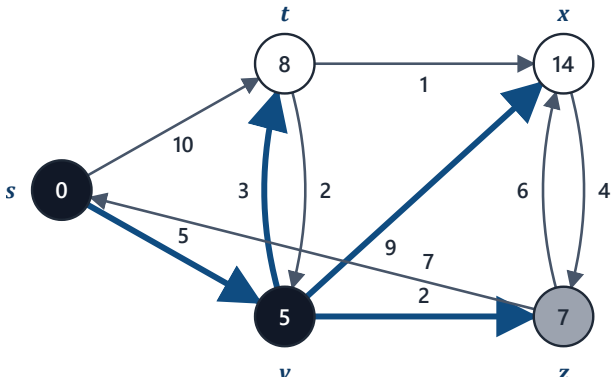
Gambar berikut memperlihatkan keadaan setelah setiap loop. Simpul hitam sudah berada di S (jaraknya final); simpul abu-abu adalah simpul dengan nilai terkecil di Q yang akan di-extract pada loop berikutnya; busur tebal adalah busur predecessor.



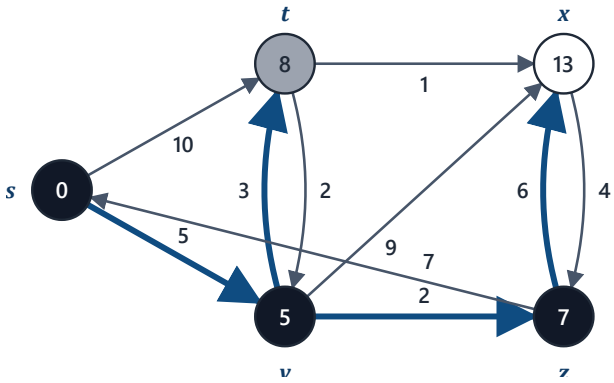
Gambar 1. (a) Setelah langkah 1–3: s.d = 0, lainnya ∞; S kosong.



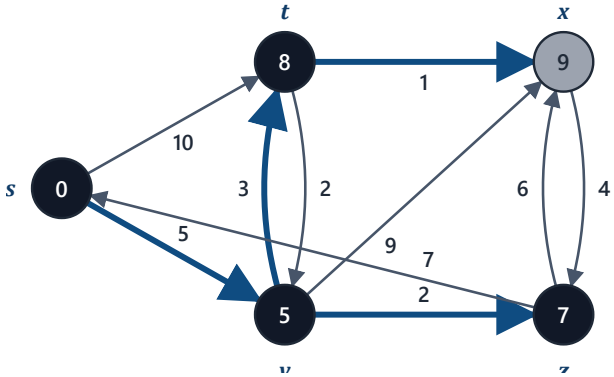
Gambar 2. (b) Loop 1: extract s; t.d=10, y.d=5.



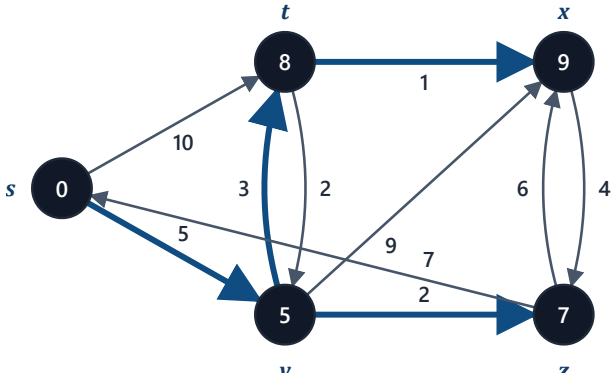
Gambar 3. (c) Loop 2: extract y; x.d=14, z.d=7, t.d=8.



Gambar 4. (d) Loop 3: extract z; x.d=13.



Gambar 5. (e) Loop 4: extract t; x.d=9.



Gambar 6. (f) Loop 5: extract x; tidak ada nilai yang berubah.

3.1 Hasil

Tujuan	Lintasan	Total bobot
t	s → y → t	$5 + 3 = 8$
x	s → y → t → x	$5 + 3 + 1 = 9$
y	s → y	5
z	s → y → z	$5 + 2 = 7$

Tabel 4. Lintasan terpendek dari s (sesuai bahan kuliah).

Perhatikan simpul x: nilainya turun tiga kali (14 → 13 → 9) sebelum di-extract. Setelah di-extract pada loop 5, nilainya tidak pernah berubah lagi. Demikian pula t: busur langsung $s \to t$ berbobot 10, tetapi lewat y

hanya 8.

3.2 Implementasi Python

Dengan `heapq`, setiap kali $v.d$ turun, pasangan baru $(v.d, v)$ dimasukkan ke heap; entri lama untuk simpul yang sudah di S diabaikan. Teknik ini setara dengan DECREASE-KEY.

Python: DIJKSTRA dengan binary heap

```python
import heapq
INF = float("inf")

def dijkstra(G, s):
    d = {v: INF for v in G}; pi = {v: None for v in G}; d[s] = 0
    S, Q = [], [(0, s)]
    while Q:
        du, u = heapq.heappop(Q)                # EXTRACT-MIN
        if u in S:
            continue                            # entri usang
        S.append(u)
        for v, w in G[u]:                       # RELAX setiap busur keluar
            if d[v] > du + w:
                d[v], pi[v] = du + w, u
                heapq.heappush(Q, (d[v], v))
    return d, pi, S

def lintasan(pi, v):
    return lintasan(pi, pi[v]) + " -> " + v if pi[v] else v

G = {"s": [("t", 10), ("y", 5)], "t": [("x", 1), ("y", 2)], "x": [("z", 4)],
     "y": [("x", 9), ("z", 2), ("t", 3)], "z": [("s", 7), ("x", 6)]}
d, pi, S = dijkstra(G, "s")
print("urutan extract:", S)
for v in "txyz":
    print(v, d[v], lintasan(pi, v))
```

Keluaran program

```
urutan extract: ['s', 'y', 'z', 't', 'x']
t 8 s -> y -> t
x 9 s -> y -> t -> x
y 5 s -> y
z 7 s -> y -> z
```

PEMBAHASAN INTI

4. Mengapa Dijkstra Benar, dan Kapan Gagal

4.1 Gagasan kebenaran

Klaim kuncinya (Cormen dkk., 2009, Teorema 24.6): ketika simpul u di-extract dan dimasukkan ke S, berlaku $u.d = \delta(s, u)$. Gagasan buktinya: misalkan ada lintasan lain dari s ke u yang lebih murah. Lintasan itu harus keluar dari S pada suatu busur (x, y) dengan $x \in S$ dan $y \notin S$. Karena x sudah final dan busurnya sudah direlaksasi, $y.d = \delta(s, y)$. Karena semua bobot **nonnegatif**, bagian lintasan dari y ke u tidak mengurangi

bobot, sehingga $\delta(s, y) \leq$ bobot lintasan itu $< u.d$. Akibatnya $y.d < u.d$, padahal u dipilih karena nilainya terkecil di Q. Kontradiksi. Jadi $u.d$ memang sudah jarak terpendek.

Langkah "karena semua bobot nonnegatif" adalah titik tempat syarat Dijkstra benar-benar dibutuhkan.

4.2 Contoh kegagalan dengan bobot negatif

CONTOH PENYANGKAL

Graf: $(s, a) = 2$, $(s, b) = 3$, $(b, a) = -2$, $(a, c) = 1$. Jarak sebenarnya: $\delta(s, a) = 3 + (-2) = 1$ dan $\delta(s, c) = 1 + 1 = 2$.

Dijkstra: extract s (a = 2, b = 3) → extract a (nilai terkecil 2), lalu RELAX (a, c) memberi $c = 3$ → extract b, RELAX (b, a) menurunkan a menjadi 1, tetapi a sudah di S dan busur keluarnya tidak pernah direlaksasi lagi → extract c dengan $c.d = 3$. Hasil akhir $c.d = 3$, padahal $\delta(s, c) = 2$. **Salah.** Untuk graf seperti ini harus memakai Bellman-Ford.

Python: Dijkstra vs Bellman-Ford pada graf berbobot negatif

```python
INF = float("inf")
G = {"s": [("a", 2), ("b", 3)], "a": [("c", 1)], "b": [("a", -2)], "c": []}

def dijkstra_buku(G, s):                    # mengikuti pseudocode: RELAX tanpa syarat
    d = {v: INF for v in G}; d[s] = 0; Q = set(G)
    while Q:
        u = min(sorted(Q), key=lambda v: d[v]); Q.remove(u)
        for v, w in G[u]:
            if d[v] > d[u] + w:
                d[v] = d[u] + w
    return d

def bellman_ford(G, s):
    d = {v: INF for v in G}; d[s] = 0
    for _ in range(len(G) - 1):
        for u in G:
            for v, w in G[u]:
                if d[v] > d[u] + w:
                    d[v] = d[u] + w
    return d

print("Dijkstra    :", dijkstra_buku(G, "s"))
print("Bellman-Ford:", bellman_ford(G, "s"))
```

Keluaran program

```
Dijkstra    : {'s': 0, 'a': 1, 'b': 3, 'c': 3}
Bellman-Ford: {'s': 0, 'a': 1, 'b': 3, 'c': 2}
```

ANALISIS DAN HUBUNGAN KONSEP

5. Running Time dan Perbandingan

5.1 Running time

Dijkstra melakukan $|V|$ kali EXTRACT-MIN dan setiap busur direlaksasi tepat sekali (karena adjacency-list setiap simpul hanya diproses saat simpul itu di-extract), sehingga ada paling banyak $|E|$ penurunan nilai (DECREASE-KEY). Running time-nya bergantung pada implementasi Q (Cormen dkk., 2009, Bab 24.3):

Implementasi Q	EXTRACT-MIN	DECREASE-KEY	Total
Array biasa (indeks simpul)	$O(V)$, sebanyak $\|V\|$ kali	$O(1)$, sebanyak $\|E\|$ kali	$O(V^2 + E) = O(V^2)$
Binary min-heap	$O(\lg V)$	$O(\lg V)$	$O((V + E) \lg V)$
Fibonacci heap	$O(\lg V)$ amortized	$O(1)$ amortized	$O(V \lg V + E)$

Tabel 5. Running time Dijkstra.

Untuk graf jarang (misalnya jaringan jalan), binary heap jauh lebih cepat daripada array. Untuk graf sangat padat, array $O(V^2)$ sudah sangat baik. Versi Fibonacci heap berasal dari Fredman dan Tarjan (1987).

5.2 Perbandingan dengan algoritma lain

Aspek	Dijkstra	Bellman-Ford (23)	BFS (17)	Prim (21)
Masalah	Lintasan terpendek berbobot	Lintasan terpendek berbobot	Lintasan dengan busur tersedikit	MST
Bobot negatif	Tidak boleh	Boleh; deteksi cycle negatif	Tanpa bobot	Boleh (tidak berpengaruh)
Struktur bantu	Antrean prioritas	-	Antrean FIFO	Antrean prioritas
Nilai yang diperbarui	$v.d = u.d + w(u, v)$	$v.d = u.d + w(u, v)$	$v.d = u.d + 1$	$v.key = w(u, v)$
Running time	$O((V + E) \lg V)$	$\Theta(VE)$	$O(V + E)$	$O(E \lg V)$

Tabel 6. Dijkstra dibandingkan dengan Bellman-Ford, BFS, dan Prim.

- **Dijkstra sebagai perumuman BFS.** Jika semua bobot 1, Dijkstra mengeluarkan simpul berdasarkan jumlah busur, persis seperti BFS. Antrean FIFO cukup untuk BFS karena semua busur berbobot sama.
- **Dijkstra dan Prim.** Strukturnya hampir identik: keduanya meng-extract simpul dengan nilai minimum lalu memperbarui tetangga. Bedanya, Prim menyimpan bobot satu busur, Dijkstra menyimpan total bobot lintasan dari s.
- **Greedy yang terbukti.** Dijkstra adalah contoh algoritma Greedy (Modul 13) yang optimal dengan syarat tertentu, dan contoh penyangkal di atas menunjukkan pentingnya syarat itu.

KESIMPULAN

6. Kesimpulan

- Dijkstra mencari lintasan terpendek dari satu sumber pada graf berbobot **nonnegatif**.
- Di setiap iterasi, simpul dengan $v.d$ terkecil di Q di-extract, dimasukkan ke S (jaraknya final), lalu semua busur keluarnya direlaksasi.
- Pada contoh bahan kuliah, urutan extract s, y, z, t, x menghasilkan $t = 8$ (s→y→t), $x = 9$ (s→y→t→x), $y = 5$ (s→y), $z = 7$ (s→y→z).
- Kebenarannya bergantung pada bobot nonnegatif; dengan bobot negatif hasilnya bisa salah, sehingga Bellman-Ford yang harus dipakai.
- Running time $O(V^2)$ dengan array, $O((V + E) \lg V)$ dengan binary heap, $O(V \lg V + E)$ dengan Fibonacci heap.

Kuis Pemahaman

Kerjakan semua soal terlebih dahulu tanpa melihat kunci jawaban. Kunci jawaban beserta pembahasannya ada di halaman berikutnya.

1 PILIHAN GANDA

Syarat utama agar Dijkstra memberikan hasil yang benar adalah ...

A. Graf harus tak berarah.
B. Semua bobot busur tidak negatif.
C. Graf tidak boleh memiliki cycle.
D. Banyaknya busur harus lebih sedikit dari banyaknya simpul.

2 PENELUSURAN

Jalankan Dijkstra dari A pada graf berarah: $(A, B) = 4$, $(A, C) = 1$, $(C, B) = 2$, $(B, D) = 1$, $(C, D) = 5$, $(D, E) = 3$. Tuliskan urutan extract, $v.d$ akhir, dan lintasan ke E.

3 ANALISIS

Pada penelusuran di modul ini, mengapa nilai $t.d$ boleh berubah dari 10 menjadi 8 pada loop 2, tetapi tidak boleh berubah lagi setelah t di-extract pada loop 4?

4 BENAR/SALAH DENGAN ALASAN

"Jika graf tidak memiliki cycle negatif, Dijkstra tetap benar walaupun ada busur berbobot negatif." Benar atau salah?

5 PERHITUNGAN

Sebuah jaringan jalan memiliki $|V| = 10.000$ persimpangan dan $|E| = 30.000$ ruas. Perkirakan banyaknya langkah Dijkstra dengan array (V^2) dan dengan binary heap ($(V + E) \lg V$). Berapa kali lebih cepat versi heap?

6 ANALISIS KASUS

Pada graf contoh modul ini, bobot busur (y, t) diubah dari 3 menjadi 6. Tentukan jarak terpendek baru ke t dan x beserta lintasannya.

7 PILIHAN GANDA

Apa perbedaan nilai yang disimpan Dijkstra dan Prim untuk setiap simpul?

A. Tidak ada perbedaan.
B. Dijkstra menyimpan total bobot lintasan dari sumber; Prim menyimpan bobot satu busur termurah ke tree.
C. Dijkstra menyimpan jumlah busur; Prim menyimpan total bobot.
D. Prim hanya bekerja pada graf berarah.

Kunci Jawaban dan Pembahasan

Bandingkan jawabanmu, lalu baca pembahasannya untuk memahami alasan di balik setiap jawaban.

Soal 1

Jawaban: B. Dijkstra dapat bekerja pada graf berarah maupun tak berarah dan pada graf yang memiliki cycle. Yang wajib hanyalah bobot nonnegatif.

Soal 2

Extract A (0): B = 4, C = 1. Extract C (1): B = $\min(4, 1+2) = 3$, D = 6. Extract B (3): D = $\min(6, 3+1) = 4$. Extract D (4): E = 7. Extract E (7). Urutan: **A, C, B, D, E**. Nilai akhir: A 0, B 3, C 1, D 4, E 7. Lintasan ke E: **A → C → B → D → E** ($1+2+1+3=7$).

Soal 3

Sebelum di-extract, $t.d$ hanyalah estimasi (batas atas) yang dapat diperbaiki bila ditemukan lintasan lebih murah, seperti lewat y. Saat t di-extract, $t.d$ adalah nilai terkecil di Q; karena semua bobot nonnegatif, lintasan lain yang melewati simpul di Q tidak mungkin lebih murah. Jadi $t.d = \delta(s,t)$ sudah final.

Soal 4

Salah. Contoh penyangkal di modul ini tidak memiliki cycle sama sekali, tetapi Dijkstra tetap menghasilkan $c.d = 3$, padahal jarak sebenarnya 2. Bobot negatif saja sudah dapat membuat Dijkstra salah.

Soal 5

Array: $V^2 = 10^8$ langkah. Binary heap: $(10.000 + 30.000) \times \lg 10.000 \approx 40.000 \times 13{,}3 \approx 532.000$ langkah. Versi heap sekitar **190 kali** lebih cepat.

Soal 6

Lintasan ke t: langsung $s \to t = 10$, atau $s \to y \to t = 5+6 = 11$. Jadi $\delta(s,t) =$ **10** lewat $s \to t$. Ke x: $s \to t \to x = 11$, $s \to y \to x = 14$, $s \to y \to z \to x = 5+2+6 = 13$. Jadi $\delta(s,x) =$ **11** lewat $s \to t \to x$.

Soal 7

Jawaban: B. Inilah satu-satunya perbedaan struktural kedua algoritma, dan perbedaan itu membuat Dijkstra menghasilkan pohon lintasan terpendek sedangkan Prim menghasilkan MST.

Referensi

Sumber utama: Materi 13.2 SINGLE SOURCE SHORTEST PATHS, Algoritma Dijkstra (bahan kuliah Perancangan dan Analisis Algoritma). Penjelasan tambahan, koreksi, dan pengayaan mengacu pada pustaka berikut.

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). Cambridge, MA: MIT Press. ISBN 978-0-262-03384-8.
2. Dijkstra, E. W. (1959). A Note on Two Problems in Connexion with Graphs. *Numerische Mathematik*, 1, 269–271. https://doi.org/10.1007/BF01386390
3. Fredman, M. L., & Tarjan, R. E. (1987). Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms. *Journal of the ACM*, 34(3), 596–615. https://doi.org/10.1145/28869.28874

4. Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Upper Saddle River, NJ: Addison-Wesley.